



Security Assessment Final Report



Safe Consensus & Coordinator

October 2026

Prepared for Safe

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	3
Assessment Methodology	4
Threat and Security Overview	5
Findings Summary	7
Severity Matrix	7
Detailed Findings	8
Critical Severity Issues	9
C-01: Missing proof of ownership for DKG encryption keys enables below threshold recovery of honest FROST shares	9
High Severity Issues	11
H-01: Epoch group substitution allows a minority validator to halt consensus	11
Medium Severity Issues	13
M-01: Identity coefficient in a DKG commitment causes validator services to exit	13
M-02: Permissionless FROST signing enables nonce sequence exhaustion and foreign group session deletion	15
M-03: New groups can accept signing requests before their first nonce commitments are usable	17
Low Severity Issues	18
L-01: Resolved complaints reset the counter used to detect compromised DKG groups	18
Disclaimer	20
About Certora	20

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Safe Consensus & Coordinator	https://github.com/safe-research/safenet L	Initial: a84cef3 Final: 7fc1e08	Solidity, EVM

Project Overview

This document describes the security review of **Safe Consensus & Coordinator**. The work was undertaken from **September 22nd, 2026** to **October 1st, 2026**.

The following contract list is included in our scope:

contracts/src/Consensus.sol

contracts/src/FROSTCoordinator.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Safenet uses a network of validators to attest to Safe transactions through FROST threshold signatures. Consensus tracks the active validator group, requests transaction attestations following oracle evaluation, and manages transitions between validator epochs. Each transition requires authorization from the current group.

FROSTCoordinator provides the on-chain coordination layer for validators to generate shared keys, commit nonces, and submit signature shares. It verifies and aggregates those shares into a group signature, which Consensus records as an attestation.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

SafeNet uses FROST threshold signatures to let a validator network attest to Safe transactions. Consensus tracks the active validator epoch, initiates transaction requests, and records attestations. FROSTCoordinator provides the on-chain coordination layer for distributed key generation, nonce preprocessing, and signature aggregation. Epoch transitions require authorization from the current validator group. Downstream Safe guards verify attestations independently; Safe owner authorization remains necessary for execution.

The expected security invariants are that epoch changes install the group authorized by the current validators, attestations authenticate the exact transaction and its context, and completed signatures remain valid and stable. Signing messages must bind the relevant epoch, oracle, oracle data, chain, and Safe transaction. Participants must be authenticated, signing nonces must never be reused, and invalid contributions must not bypass cryptographic verification. Availability additionally requires recovery from minority validator failures without incorrectly excluding honest participants.

The principal trust boundary lies between permissionless requests and validator-authorized outcomes. Public callers can initiate coordinator activity, while participant contributions are associated with authenticated addresses and group membership. Some DKG validation and transaction policy enforcement occur in the validator software, so coordinator events alone do not establish that a ceremony is trustworthy. Security assumes fewer than a signing threshold of validators are compromised, sufficient honest participants remain available, configured oracles provide trustworthy verdicts, and validators process chain events correctly. Downstream acceptance of historical group keys also makes protection and retirement of old secret shares important.

The review considered malicious public callers and participating validators, including crafted DKG inputs, group substitution, signature and message replay, nonce misuse, callback manipulation, withheld contributions, and request flooding. Consensus and FROSTCoordinator do not directly custody transaction funds, but failures can prevent attestations or weaken the authorization relied upon by downstream consumers. Economic exposure includes wasted transaction fees, preprocessing costs, and disruption of fee-funded oracle requests. Dependency surfaces

include curve arithmetic, hashing, Merkle proofs, validator event handling, oracle calls, and cross-chain attestation verification.

Manual review covered both contracts, their supporting cryptographic and state-management libraries, and relevant validator, oracle, and guard interactions. The checks focused on authorization, epoch transitions, message binding, signature validity, nonce allocation, callback behavior, and recovery paths. No tests were written or executed, and no fuzzing or formal verification was performed as part of this review.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	1	1	1
High	1	1	1
Medium	3	3	3
Low	1	1	1
Informational	-	-	-
Total	6	6	6

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
Low	Low	Low	Medium	Medium
Likelihood				

Detailed Findings

ID	Title	Severity	Status
C-01	Missing proof of ownership for DKG encryption keys enables below threshold recovery of honest FROST shares	Critical	Fixed
H-01	Epoch group substitution allows a minority validator to halt consensus	High	Fixed
M-01	Identity coefficient in a DKG commitment causes validator services to exit	Medium	Fixed
M-02	Permissionless FROST signing enables nonce sequence exhaustion and foreign group session deletion	Medium	Fixed
M-03	New groups can accept signing requests before their first nonce commitments are usable	Medium	Fixed
L-01	Resolved complaints reset the counter used to detect compromised DKG groups	Low	Fixed

Critical Severity Issues

C-01: Missing proof of ownership for DKG encryption keys enables below threshold recovery of honest FROST shares

Severity: Critical	Impact: High	Likelihood: Medium
Files: /contracts/src/FROSTCoordinator.sol	Status: Fixed	

Description:

In `keyGenCommit` the contract treats `q` like a valid encryption address but it never checks that the participant actually owns the secret key behind that address so suppose attacker C copies honest A's `q` and attacker D copies honest B's `q`. When an honest sender X publishes encrypted shares, X uses the same ECDH/XOR pad for the honest recipient and the attacker that copied that recipient's `q`.

C and D can complain about X and receive the plaintext shares intended for themselves through `keyGenComplaintResponse` and XOR plaintext with ciphertext to recover the pads. They can then decrypt X's ciphertexts intended for A and B. In a 4 of 7 group the attackers now have four points from each sender's polynomial, at the two colluder IDs and the two copied honest IDs so they can reconstruct the polynomial, recover the missing self evaluation and build real signing shares. The final signature check passes because the recovered material is genuine key material.

Recommendations:

Require every DKG participant to provide a separate proof of possession for `q`, using the same `c[0]` proof scheme with the challenge bound to the participant's FROST identifier and `q`.

Customer Response:

Fixed in [PR #1027](#).

Fix Review:



Fix confirmed.

High Severity Issues

H-01: Epoch group substitution allows a minority validator to halt consensus

Severity: High	Impact: High	Likelihood: Medium
Files: contracts/src/Consensus.sol	Status: Fixed	

Description:

An epoch rollover signature authorizes the successor's public key but does not bind the successor's FROST group ID. `Consensus.stageEpoch` nevertheless installs a caller-supplied group ID as the group responsible for the new epoch. An attacker can create a different finalized group with the same public key and substitute its ID while reusing a valid rollover signature.

The coordinator does not verify DKG proofs of knowledge onchain. An attacker-controlled group can therefore finalize commitments that sum to the legitimate successor's public key while registering unrelated verification shares, without knowing the corresponding private key. Honest validators' offchain DKG checks do not prevent this because they validate the legitimate successor group, not the substituted group.

A malicious validator included in the active signing selection (for example, one out of four validators, within the network's stated fault tolerance) can execute the substitution by submitting the final valid signature share with a modified staging callback. Honest validators sign the legitimate rollover message and need not make any mistake. The substituted group can exclude all legitimate successor validators and have no usable signing shares, persistently halting normal transaction attestation and further epoch rotation.

An attack sequence would look as follows:

1. A legitimate successor group `G` completes DKG with public key `Q`. The active validators prepare a rollover to epoch `e` at block `b`.
2. The attacker creates and finalizes `G_prime`, containing attacker-controlled addresses and the same public key `Q`, but unusable signing shares.

3. Honest active validators submit valid shares for the legitimate rollover message. The malicious selected validator waits until its share will complete the signature.
4. Before the rollover block and signing timeout, the malicious validator calls `signShareWithCallback` with its valid share, the correct selection and proof, and a callback targeting `Consensus.onSignCompleted`. The callback context supplies `G_prime` instead of `G` for staging.
5. The coordinator verifies and completes the genuine active-group signature, then invokes the caller-supplied callback in the same transaction.
6. `stageEpoch` retrieves `Q` from `G_prime`, verifies the unchanged signed message, and stores `$groups[e] = G_prime`.
7. At block `b`, `G_prime` becomes the active group.

The staged rollover cannot be replaced or cancelled, and subsequent rollovers require a signature completed through the substituted group ID. Signatures from the legitimate group cannot repair the mapping, even though its public key matches. Honest validators therefore cannot restore progress through the normal rollover flow.

Safe owners retain the guard's delayed announcement escape hatch. The impact is persistent disruption of normal consensus operation; this attack does not establish signature forgery, theft, or permanent freezing of Safe funds.

Recommendations:

Include the successor group ID alongside its public key in the signed `EpochRollover` message, and update all message producers and verifiers consistently.

Customer Response:

Fixed in [PR #1022](#).

Fix Review:

Fix confirmed.

Medium Severity Issues

M-01: Identity coefficient in a DKG commitment causes validator services to exit

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: contracts/src/FROSTCoordinator.sol	Status: Fixed	

Description:

A malicious participant in a legitimate DKG ceremony can submit a commitment containing the identity point $(0,0)$ as a higher-order coefficient, such as $c[1]$, alongside a valid encryption key, constant commitment, and proof of knowledge. Both `keyGenCommit()` and the Rust commitment verification accept this input.

The validator stores the accepted commitment in its state, but the pinned FROST serializer rejects identity elements. Snapshot serialization therefore fails, and the service driver treats the error as unrecoverable and exits. The root cause is a mismatch between accepted in-memory values and their persistent encoding.

One malicious DKG participant can consequently stop affected validators from processing subsequent events, including current-epoch signing requests. If enough signers are affected, transaction attestations become unavailable. Restarting can replay the offending event and reproduce the failure. The attack requires one commitment transaction and no invalid proof or completion of the ceremony; no key recovery or signature forgery is established.

Recommendations:

Ensure accepted and derived commitments can be safely serialized. Reject unsupported identity coefficients before inserting them into state, or provide a compatible persistent encoding. Also handle identity coefficients arising from aggregation; checking individual inputs alone is insufficient. Unsupported values should cause recoverable validation or ceremony failures rather than terminate the service.

Customer Response:

Fixed in [PR #1025](#).

Fix Review:

Fix confirmed.

M-02: Permissionless FROST signing enables nonce sequence exhaustion and foreign group session deletion

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: contracts/src/FROSTCoordinator.sol	Status: Fixed	

Description:

FROSTCoordinator.sign is permissionless and uses one sequence for the entire group including requests unrelated to approved Consensus messages. In the rust validator's `handle_sign` event handler, the validator observes the event's sequence and mutates NonceState before confirming that the message and group match a pending signing request. `NonceState::observe` advances the sequence window and prunes older nonce chunks even when no usable nonce exists.

An attacker can therefore repeatedly call `sign` on the production group with unrelated messages and consume nonce positions faster than validators can preprocess replacements. Eventually a legitimate transaction or rollover reaches a sequence for which validators have no linked nonce causing its pending `WaitingForRequest` state to be removed. The same mutation order enables a more targeted attack. An attacker can finalize a separate attacker controlled group and call sign with the victim's pending message. Because validators have no nonce state for the foreign group, nonce lookup returns nothing but `handle_sign` has already removed the message keyed pending request and does not restore it. When the legitimate production group Sign event arrives, the validator no longer has the session required to continue.

Recommendations:

Restrict `sign` for production groups to authorized callers. In the validator match the pending message and group before mutating nonce state or removing the request, and retain the request if no linked nonce is available.

Customer Response:

Fixed in [PR #1029](#).

Fix Review:

Fix confirmed.

M-03: New groups can accept signing requests before their first nonce commitments are usable

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: contracts/src/FROSTCoordinator.sol	Status: Fixed	

Description:

The coordinator allows signing as soon as a group is finalized, but the Rust validators generate and publish that group's first nonce commitments only afterward. An attacker can call `Consensus.proposeTransaction` during this gap, including immediately after a new epoch activates. The proposal creates a signing sequence and a one-shot oracle request. When validators handle the `Sign` event, they have no linked nonce for that sequence and remove the pending signing request. Publishing nonce commitments later cannot recover the consumed sequence, and the oracle will not create the same request again. The exact transaction request can therefore remain unattested even though the group has enough honest validators to sign.

Recommendations:

Replace the advance `preprocess` flow with nonce commitments submitted by validators for each signing ceremony.

Customer Response:

Fixed in [PR #1029](#).

Fix Review:

Fix confirmed.

Low Severity Issues

L-01: Resolved complaints reset the counter used to detect compromised DKG groups

Severity: Low	Impact: Low	Likelihood: Low
Files: contracts/src/FROSTCoordinator.sol	Status: Fixed	

Description:

`keyGenComplain()` compares a participant's outstanding accusations against the signing threshold. However, `FROSTParticipantMap.respond()` decrements that same counter whenever a response is submitted. The Rust validator instead checks cumulative accusations and never decreases that total.

Consequently, resolving complaints between submissions allows the contract to miss the cumulative threshold. For example, with threshold four, answering three complaints resets the contract's counter to zero; a fourth complaint increases it to one and emits `compromised = false`, while validators reject the ceremony.

This permits contract/validator state disagreement and, where sufficient confirmations were already submitted, allows a rejected ceremony to finalize onchain. The current Rust validator independently aborts before producing the threshold response, so this mismatch alone does not establish polynomial recovery from an honest validator's responses.

Recommendations:

Do not decrement the accusations for the accused.

Customer Response:

Fixed in [PR #1019](#).

Fix Review:



Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.